EnergyWatch AI

INCIDENT

Rapport d'incident INC-0001

Post-mortem de l'exercice de panne du fournisseur LLM : chronologie, étapes du runbook appliquées, preuves et améliorations.

Projet	**EnergyWatch AI, plateforme multi-agents (LangGraph)**
Secteur	**Énergie**
Cadre	**Mini-projet S5 · Agent en production**
Version	**1.2.0**
Date	**30 octobre 2026**
Dépôt	**github.com/bouincha/Energy-Watch-AI**

Rapport d'incident INC-0001 : panne du fournisseur LLM (exercice)

Champ	Valeur
Incident	INC-0001
Runbook	RB-001 : panne du fournisseur LLM
Gravité	SEV-2
Type	Exercice planifié (injection de panne), réalisé sur l'image Docker de production
Date	30/10/2026
Détecté	11:16:00.918 UTC (automatiquement, par l'agent de monitoring)
Résolu	11:16:32.064 UTC (automatiquement, par l'agent de monitoring)
MTTR	**31,15 s**
Exécutions touchées	`drill-outage-0001` (et `drill-outage-0002`, servie en mode dégradé pendant l'ouverture du disjoncteur)
Perte de données	Aucune
Impact utilisateur	Les réponses sont restées disponibles, marquées « dégradées » : pas de texte rédigé par l'IA, mais tous les chiffres étaient exacts

Résumé

Pour vérifier que le runbook RB-001 fonctionne réellement, nous avons simulé une panne totale du fournisseur LLM avec le contrôleur de chaos (`POST /api/chaos {"llm_outage": true}`) sur l'application conteneurisée (`energywatch-ai:local` , `ADMIN_API_KEY` défini). Le disjoncteur s'est ouvert après 3 échecs consécutifs, l'agent de monitoring a ouvert INC-0001, les agents sont passés sur leurs replis déterministes, et l'incident s'est résolu tout seul une fois la panne terminée et le délai de refroidissement du disjoncteur écoulé.

Chronologie (UTC, d'après l'incident enregistré et les logs du conteneur)

Heure	Événement	Correlation ID
11:15:59	L'opérateur active `llm_outage` (audité : `chaos.updated`)	-
11:15:59	Lancement de « Give me the full monthly report » ; les appels LLM échouent (`ChaosError`)	`drill-outage-0001`

Heure	Événement	Correlation ID
11:16:00.918	3e échec consécutif : disjoncteur `closed → open`	`drill-outage-0001`
11:16:00.918	**Ouverture d'INC-0001** (Détecter) et mode dégradé appliqué (Atténuer)	`drill-outage-0001`
11:16:00	Exécution terminée `degraded` en 1 871 ms : 3 appels LLM, 3 échecs, réponse livrée	`drill-outage-0001`
11:16:01	2e exécution terminée `degraded` en **54 ms** : 0 appel LLM (court-circuité par le disjoncteur ouvert)	`drill-outage-0002`
11:16:01	`/ready` indique `status=degraded`, `circuit_breaker=open`, `open_incidents=[INC-0001]`	-
11:16:01	L'opérateur désactive `llm_outage` (fin de la panne simulée)	-
11:16:31	Fin du refroidissement (30 s) : disjoncteur `half_open`	-
11:16:32.064	Appel de test réussi : disjoncteur `half_open → closed`, **INC-0001 résolu**	`drill-recovery-0001`

Extrait des logs (`docker logs ewa-test | grep -E "circuit breaker|incident"`) :


```
{"ts": "2026-10-30T11:16:00.918Z", "level": "WARNING", "logger": "energywatch.resilience",
"msg": "circuit breaker 'llm' closed → open", "correlation_id": "drill-outage-0001"}
{"ts": "2026-10-30T11:16:00.918Z", "level": "ERROR", "logger": "energywatch.incidents", "msg":
"incident INC-0001 opened: LLM provider outage", "correlation_id": "drill-outage-0001",
"runbook": "RB-001", "severity": "SEV-2"}
{"ts": "2026-10-30T11:16:32.064Z", "level": "INFO", "logger": "energywatch.resilience", "msg":
"circuit breaker 'llm' half_open → closed", "correlation_id": "drill-recovery-0001"}
{"ts": "2026-10-30T11:16:32.064Z", "level": "WARNING", "logger": "energywatch.incidents", "msg":
"incident INC-0001 resolved", "correlation_id": "drill-recovery-0001", "mttr_s": 31.15,
"resolved_by": "monitoring-agent"}
```


Étapes du runbook appliquées

Étape RB-001	Appliquée ?	Preuve
1. Détecter	✅ automatique	Entrée « Detect » dans la chronologie de l'incident, log `incident INC-0001 opened`
2. Atténuer (mode dégradé)	✅ automatique	Spans des agents LLM `degraded` ; réponse avec la mention dégradée ; 5 anomalies toujours signalées
3. Communiquer	✅ automatique	`/ready` = degraded ; barre latérale `LLM: … · open` ; incident visible dans Monitoring
4. Diagnostiquer	✅	Les logs indiquent `error_type=chaos` pour chaque appel en échec, ce qui confirme la cause
5. Rétablir (test en semi-ouvert)	✅ automatique	Entrée « Recover : half-open probe succeeded »

Étape RB-001	Appliquée ?	Preuve
6. Résoudre	✅ automatique	Entrée « Resolve », MTTR de 31,15 s

Ce qui a bien fonctionné

- **Aucune interruption pour l'utilisateur :** les deux exécutions ont renvoyé des réponses complètes et exactes, car les chiffres proviennent d'outils déterministes.
- **Échec rapide :** une fois le disjoncteur ouvert, les exécutions prenaient 54 ms au lieu d'attendre un fournisseur hors service.
- **Traçabilité :** une seule recherche sur le correlation ID suffit pour reconstituer tout l'incident.

Pistes d'amélioration

1. **Latence de la première exécution en échec (1,9 s) :** chaque appel en échec est retenté avant l'ouverture du disjoncteur. *Action :* garder `LLM_MAX_RETRIES=2` (utile pour les erreurs passagères) mais rendre le délai entre tentatives configurable.
2. **Le MTTR dépend surtout du refroidissement de 30 s.** C'est acceptable pour un SEV-2 ; un délai plus court multiplierait les appels de test vers un fournisseur en panne.
3. **Télémétrie en mémoire :** les incidents sont perdus au redémarrage (les logs JSON restent). *Action (prochaine version) :* exporter les métriques et incidents vers un stockage externe.

Reproduire l'exercice

```
curl -X POST $URL/api/chaos -H "X-API-Key: $ADMIN_API_KEY" -H "Content-Type: application/json" -
d '{"llm_outage": true}'
curl -X POST $URL/api/agent/ask -H "Content-Type: application/json" -d '{"query": "Give me the
full monthly report"}'
curl $URL/api/incidents
curl -X POST $URL/api/chaos -H "X-API-Key: $ADMIN_API_KEY" -H "Content-Type: application/json" -
d '{"llm_outage": false}'
sleep 31 && curl -X POST $URL/api/agent/ask -H "Content-Type: application/json" -d '{"query":
"Any anomalies at the plant?"}'
curl $URL/api/incidents   # INC-0001 est maintenant résolu, avec son MTTR
```

Le même scénario est vérifié automatiquement par le test `tests/test_incidents.py::test_llm_outage_incident_lifecycle`.